LAB 4 (Part#2)
Secondary Data Structures
[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python template**.
- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 3 Assignment Tasks: 3
Total Marks: 15

Intro Lab Tasks [No Need to Submit]

Implementing an Array-Based Stack

A stack stores elements such that the most recently inserted element is the first one removed. This follows the **Last-In, First-Out (LIFO)** principle. The **ArrayStack** class has only **two private instance variables:**

- An array of Objects called **'stack'** (You take the size using the constructor).
- An integer called **'top'** which initializes at -1. The top index increases when something is "pushed" and it decreases when something is "popped".

Complete the **ArrayStack** class by implementing the following methods:

Methods	Description
push(element)	Inserts an element at the top of the stack. If the stack is full, just print stack overflow. The push always increments the top.
pop()	Removes and returns the top element of the stack. If the stack is empty, just print "stack underflow". The pop() always decrements the top.
peek()	Returns the top element without removing it. If the stack is empty, return null.
isEmpty()	Returns true if the stack contains no elements, otherwise false.

Implementing an Array-Based Queue

A queue stores elements such that the earliest inserted element is the first one removed. This follows the **First-In, First-Out (FIFO)** principle. The **ArrayQueue** class has the following **four private instance variables**:

- An array of Objects called **'queue'** (You take the size using the constructor)
- Three integers:
  - **'front'** – indicates the index of the element at the front of the queue.
  - **'rear'** – indicates the index where the next element will be inserted.
    - [Both front and rear should initialize to 0]
  - **'size'** - indicates the number of elements in the queue

Complete the **ArrayQueue** class by implementing the following methods:

Methods	Description
enqueue(element)	Inserts an element at the rear of the queue. If the queue is full, print "queue overflow". The enqueue always increments the rear. When you reach the end, the value of rear rounds back up to the first index.
dequeue()	Removes and returns the front element of the queue. If the queue is empty, print "queue underflow" (or throw/raise an Exception). The dequeue always increments the front. When you reach the end, the value of rear rounds back up to the first index.
peek()	Returns the front element without removing it. If the queue is empty, return null.
isEmpty()	Returns true if the queue contains no elements, otherwise false.

Lab Tasks [No Need to Submit]

1. Stack: Tower of Blocks

A kid is trying to make a tower using blocks with numbers or characters written on them. He puts a block on the top of another to make the tower. As for removing the blocks, he removes the topmost block if needed. He is afraid that removing other blocks from the middle in one go will result in the collapse of his tower.

Now his friend wants your nth block from the top of your tower. He is willing to give him the nth block from the top, preserving the relative position of others.

Following his tower build process, solve this scenario using **Stack** class. You can only use the stack methods.

In the following examples, consider the rightmost element to be the topmost element of the stack. **(Rightmost item is the top)**

Sample Tower	Sample Output	Explanation
Stack: 4 19 23 17 5 n: 2	Stack: 4 19 23 5	In this tower, the 2nd block from the top is the block with 17. After removing the block the stack looks like 4 19 23 5
Stack: 73 85 15 41 n: 3	Stack: 73 15 41	In this tower, the 3rd block from the top is the block with 85. After removing the block the stack looks like 73 15 41

[NOTE: A LinkedListStack class is already given and you can create an object of Stack to call push(),pop(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Stack

2. <u>Queue</u>: Remove Consecutive Duplicates

NOTE: YOU MUST USE QUEUE TO SOLVE THIS TASK

Write a method/function called **removeConsecutiveDups()** that takes only one String in its parameter. You are given a String which may or may not contain consecutive duplicate values. Finally the **removeConsecutiveDups()** method returns a new String containing no consecutive duplicate characters. If no duplicates are found then it simply returns the same string.

Sample Given String	Sample Returned String
`aabbbccccdd`	abcd
`aaabbaa`	aba
`abcabcabc`	abcabcabc
`aaaaa`	a

[Hint: A LinkedListQueue class containing enQueue(), deQueue(), peek() and isEmpty() methods is already given in the driver code, you just need to create an Object of it to solve the task. No need to design a new Queue]

You cannot use any data structure other than Queue

Assignment Tasks [Need to Submit]

1. Stack: Conditional Reverse

Consider that a Stack class has been implemented containing the push(element), pop(), peek() and isEmpty() functions.

Complete the function **conditional_reverse()** which will take an object of Stack class as an input that contains some integer values. **The function returns a new stack** which will contain the values in reverse order from the given stack **except the consecutive one's.** You cannot use any other data structure except Stack.

In the following example, consider the rightmost element to be the topmost element of the stack.

Sample Input Stack (Rightmost is the top)	Returned Reversed Stack (Rightmost is the top)
Stack: 10, 10, 20, 20, 30, 10, 50 **Top** = 50	**Stack:** 50, 10, 30, 20, 10 **Top** = 10
Explanation: Consecutive 20 and 10 are not present in the output reversed stack	

[NOTE: A LinkedListStack class is already given and you can create an object of Stack to call push(),pop(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Stack

2. <u>Queue</u>: Hot Potato Game

A group of friends are standing in a circle playing the **Hot Potato game**. The students pass an imaginary "potato" to the next person in the circle. After the potato has been passed a fixed number of times k, the student holding the potato is eliminated from the game. The process then continues with the remaining students, starting from the next student in the circle. This continues until only one student remains, who is declared the winner.

Write a method **hotPotato(players, k)** that simulates this game using a queue. The method takes an array of Strings (players) and an integer (k). The method should repeatedly rotate the queue to simulate passing the potato and remove the player who is eliminated. Finally, the method should **return** the name of the winning player.

Sample Input	Sample Output
players = ["Ali", "Ben", "Cia", "Dan", "Eli", "Faye"] k = 3	Dan eliminated Ben eliminated Ali eliminated Cia eliminated Faye eliminated Winner: Eli
<u>Explanation:</u> **Queue initially:** Ali Ben Cia Dan Eli Faye Passing 3 times → Dan eliminated → Eli Faye Ali Ben Cia Passing 3 times → Ben eliminated → Cia Eli Faye Ali Passing 3 times → Ali eliminated → Cia Eli Faye Passing 3 times → Cia eliminated → Eli Faye Passing 3 times → Faye eliminated → Eli **Remaining player → Eli (Winner)**	
players = ["Ali", "Rafi", "Sara", "Nina"] k = 2	Sara eliminated Rafi eliminated Nina eliminated Winner: Ali

[NOTE: A LinkedListQueue class is already given, you can use it to create an Object of Queue to call enqueue(),dequeue(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Queue

3. <u>Stack and Queue</u>: Dancing Pairs

From a dance school, you are given a **Stack filled with Dancer's Info (Objects of Dancer class)**. The Dancer Class only has three instance variables; name(String), gender (char), id (int). The genders are either "M" or "F" (male or female) and the id can be any random integer.

The stack of dancers need to be paired off as male with female for a competition. But this pairing has be done by following the rules of the competition:

- If ***a male & a female*** or ***a female & a male*** are consecutive in the stack, they get paired immediately and popped out of the Stack since they've been processed.
- If two consecutive females are found then the ***first spare female*** should be popped out of Stack and enqueued in a **femaleQueue** for future pairing.
- If two consecutive males are found, we try to pair the ***first spare male*** with a spare female from the **femaleQueue**. If no spare female is available in the then the spare male is enqueued in a **maleQueue**.
- Once all the dancers have been processed from the Stack, the dancers in the **maleQueue** or **femaleQueue** need to be processed, in case they're not empty. If a dancer couldn't be paired up they need to be mentioned at the very end.

Your task is to complete the given method **dance_pair()** that takes **a stack** and prints every male-female pair and those who couldn't be paired as well.

NOTE: A LinkedListStack & LinkedListQueue class is already given, you can use it to create an Object of Stack to call push(),pop(),peek(),isEmpty() and Queue to call enqueue(),dequeue(),peek(),isEmpty() to complete the task.

In both classes, isEmpty() returns true/false, peek() and pop() returns null/None for underflow.

You cannot use any data structure other than stack and queue

sample examples are given on the next page

Sample Given Stack	Sample Output
Nina(F-12) <- TOP Omar(M-44) Liam(M-24) Sara(F-15) Maya(F-54) Arif(M-10)	#1 Nina(F-12) & Omar(M-44) #2 Liam(M-24) & Sara(F-15) #3 Maya(F-54) & Arif(M-10)
Rita(F-43) <- TOP Mina(F-54) Lina(F-12) Omar(M-9) Arif(M-51) Sami(M-53) Nina(F-29) Lara(F-43) Tariq(M-45)	#1: Lina(F-12) & Omar(M-09) #2: Arif(M-51) & Rita(F-43) #3: Sami(M-53) & Nina(F-29) #4: Lara(F-43) & Tariq(M-45) Unpaired Female: Mina(F-54)
Sara(F-43) <- TOP Lina(F-52) Omar(M-16) Rafi(M-23) Maya(F-65) Alim(M-11) Kazi(M-32) Nina(F-75) Anan(M-11) Wafi(M-13)	#1: Lina(F-52) & Omar(M-16) #2: Rafi(M-23) & Maya(F-65) #3: Alim(M-11) & Sara(F-43) #4: Kazi(M-32) & Nina(F-75) Unpaired Male: Anan(M-11), Wafi(M-13)